

PureScript .NET Backend Feasibility

Status: active **Category:** research **Created:** 2026-02-20 **Author:** Claude (research session)

Executive Summary

Compiling PureScript to the .NET CLR is **technically feasible** and architecturally sensible, but **no one has attempted it**. There is no existing PureScript-.NET backend, not even an abandoned prototype.

The CLR is a reasonable target for PureScript's core semantics: native tail-call support, well-understood encodings for algebraic data types (proven by F#), and type classes/HKTs are non-issues since PureScript erases them before CoreFn. The main challenges are currying overhead (delegate allocations), the FFI rewrite cost, and the small community intersection between PureScript and .NET developers.

Appealing use cases include embedding PureScript into Excel (via COM/.NET interop), Azure Functions (serverless), Unity (games), and enterprise environments locked into the Microsoft stack.

CLR Fit for PureScript Features

Good Fit

Feature	How it maps to .NET
Algebraic data types	F# proves the encoding: abstract base class + subclass per constructor + tag. Well-understood, efficient.
Pattern matching	Tag switches / conditional chains. Straightforward.
Tail call optimization	CLR natively supports TCO via <code>.tail</code> IL prefix instruction (ECMA-335). Ahead of Python/Lua; on par with BEAM for simple cases.
Type classes	Non-issue. Desugared to dictionary-passing before CoreFn. Backend never sees them.
Higher-kinded types	Non-issue. Erased before CoreFn.
Immutability	.NET has <code>System.Collections.Immutable</code> ; F# defaults to immutable. Not as mature as Haskell's persistent structures but workable.

Challenging

Feature	Difficulty
Curried functions	Pervasive currying means many <code>Func<></code> delegate allocations. Each closure = 1-2 heap allocs. <code>purescript-backend-optimizer</code> uncurrying pass is essential.

Feature	Difficulty
Row-polymorphic records	No natural .NET representation. Would need dictionaries or generated classes.
Effect/Aff	Effect thunks → Func<T> (allocation overhead). Aff would need reimplementation atop Task<T> / ValueTask<T> .
Data representation	PureScript's JS backend uses plain objects. .NET needs proper class hierarchies — more rigid, more verbose.

Comparison with Existing Backends

Dimension	Lua	Erlang/BEAM	Python	.NET
TCO	Manual trampolining	Native (built for it)	None	Native (.tail IL prefix)
Closures	Lightweight	Lightweight	Lightweight	Heap-allocated delegates
Currying overhead	Low (dynamic)	Low (dynamic)	Low (dynamic)	Moderate-high
Ecosystem	Small	Medium (OTP)	Huge (ML/AI)	Huge (enterprise)
Performance	Moderate (LuaJIT fast)	Good concurrent	Slow	Excellent (JIT + NativeAOT)
Target typing	Dynamic	Dynamic	Dynamic	Static
Deployment	Embedded, games	Distributed systems	Scripts, ML	Azure, Unity, desktop, mobile, WASM

Precedent: Functional Languages on .NET

Project	Status	Lessons
F#	Production, Microsoft-maintained	Proves ML-family works well on CLR. Co-developed with CLR team though — not an external effort.
ClojureCLR	Active (ClojureCLR-Next rewrite underway)	Most successful non-Microsoft FP-on-.NET. Uses Dynamic Language Runtime.
SML.NET	Defunct (~2006)	Proved ML → CLR compilation works.
Eta (Haskell → JVM)	Stalled	Showed lazy pure FP can target managed runtimes, but maintenance killed it.

Project	Status	Lessons
Haskell → .NET	Never succeeded	Multiple research attempts (Canterbury 2002, UFPE), none production. Wiki says "short answer is you can't."
rustc_codegen_clr	Experimental (2024)	Rust → CIL, runs 95% of core tests. Even very different paradigms can target CLR.
Scala.NET	Never completed	
ILX (MS Research)	Research, fed into F#	Extended CIL with closures, ADTs, generics. Techniques now used by F#.

Recommended Architecture

```
PureScript source
  → purs compile --dump-corefn
  → purescript-backend-optimizer (inlining, uncurrying, DCE)
  → .NET code generator (reads optimized CoreFn + externs.json for
types)
  → C# source files (or CIL bytecode)
  → dotnet build (or ilasm)
  → .NET assembly (.dll)
```

This mirrors `purescript-backend-erl`, the most mature alternative backend.

Key Design Decisions

Code generation target: Two options with different tradeoffs.

- **Emit C# source** — Simpler to implement, easier to debug, leverages Roslyn optimizations. But Roslyn *never* emits `.tail`, so no TCO beyond what PureScript's own pass handles (self-recursion only, not mutual recursion).
- **Emit CIL directly** — Full control over tail calls, data layout, closures. Significantly harder to implement. Could use `ilasm` or `System.Reflection.Emit`.

Type recovery: Unlike dynamic targets, .NET benefits from type information. The backend should read `externs.json` to generate properly typed IL/C# rather than boxing everything as `object`.

FFI language: C# is the natural choice (lingua franca of .NET). F# would also work but adds FSharp.Core dependency.

Constructor representation: F#-style class hierarchies: abstract base + subclass per constructor + integer tag. Aligns with CLR JIT virtual dispatch.

Unique Value Proposition

What makes .NET interesting beyond "yet another backend":

1. **Excel embedding** — COM interop / .NET for Office add-ins. PureScript logic driving spreadsheet calculations.
 2. **Unity** — C#/Mono runtime. PureScript for game logic with Unity rendering.
 3. **Azure Functions** — Serverless PureScript with the same type-safe code as the frontend.
 4. **Enterprise access** — Opens PureScript to organizations locked into Microsoft stacks.
 5. **NativeAOT** — Ahead-of-time compilation to native binaries. Potentially the fastest PureScript backend.
 6. **Cross-backend sharing** — Same PureScript business logic on JS (browser), .NET (server/Excel), Erlang (distributed).
-

Why It Hasn't Been Done

1. **Small community intersection** — PureScript devs live in JS; .NET FP devs already have F#.
 2. **FFI rewrite cost** — Every core library (Prelude, Effect, Aff, ST, Refs) needs .NET FFI from scratch. This is the largest single effort for any new backend and the main reason most alternative backends die.
 3. **Maintenance burden** — CoreFn changes with compiler releases. Most alternative backends (Python, Kotlin, Swift, Clojure) have gone stale.
-

Assessment

Feasibility: High. The CLR is well-suited and the architecture is proven via `purel`.

Effort: Large. Comparable to `purel`, which took a dedicated team (`id3as`) with production motivation.

Bang-for-buck: Moderate. The deployment breadth is unmatched, but F# already serves most ".NET + FP" needs. The case is strongest for specific niches: Excel embedding, Unity, cross-backend PureScript codebases, and the "purity guarantees beyond what F# offers" argument.

Recommended next step if pursued: Prototype emitting C# from CoreFn for a minimal subset (Prelude + Effect), using `purescript-backend-optimizer` and `purescript-backend-erl` as architectural references.